

PAKISTAN CYBER EMERGENCY RESPONSE TEAM

VULNERABILITY ASSESSMENT INTERNSHIP

ASSIGNMENT 6

VAPT Assessment Report



PREPARED BY

Amna Imtiaz

PREPARED FOR

VA Internship Batch

Date: August 19, 2026

VAPT Assessment Report

Target Application: OWASP Juice Shop

1. EXECUTIVE SUMMARY

Report Title: Web Application Vulnerability Assessment and Penetration Testing (VAPT) Report

Target Application: OWASP Juice Shop (v15.0.0)

Testing Environment: Local Hosted Instance — <http://localhost:3000>

Assessment Date: 19 August 2026

Prepared By: Amna Imtiaz

Report Classification: Confidential

Overall Risk Rating: CRITICAL



1.1 Summary Overview

A comprehensive Vulnerability Assessment and Penetration Testing (VAPT) engagement was conducted against the **OWASP Juice Shop** web application — an intentionally vulnerable application designed specifically for security training and testing purposes. This assessment was performed in a **controlled, authorized environment** following industry-standard methodologies.

The primary objective was to identify, exploit, and document security vulnerabilities present within the web application, and to provide actionable remediation recommendations aligned with **OWASP Top 10** guidelines.

1.2 Key Findings Summary

Risk Level	Count	Percentage
 Critical	3	25%
 High	4	33%
 Medium	3	25%
 Low	2	17%
Total	12	100%

1.3 Critical Observations

During the assessment, the following **critical security issues** were identified:

- **SQL Injection** vulnerabilities allowing complete authentication bypass
- **Cross-Site Scripting (XSS)** enabling malicious script execution
- **Broken Access Control** exposing unauthorized admin functionalities
- **Sensitive Data Exposure** through unprotected API endpoints
- **Security Misconfiguration** in application headers and error handling

2. ASSESSMENT SCOPE & OBJECTIVES

2.1 Scope of Assessment

Parameter	Details
Application Name	OWASP Juice Shop
Application Type	Web Application (Node.js / Angular)
Testing URL	http://localhost:3000
Testing Type	Black Box + Grey Box Testing
Environment	Local Isolated Lab Environment
Authorization	Explicitly Authorized (OWASP Open-Source Training App)

2.2 Objectives

The following objectives were defined for this assessment:

1. **Identify** security vulnerabilities within the web application
2. **Exploit** identified vulnerabilities in a controlled manner to validate their existence
3. **Assess** the potential business impact of each vulnerability
4. **Document** all findings with proof-of-concept evidence
5. **Recommend** practical remediation strategies for all identified issues

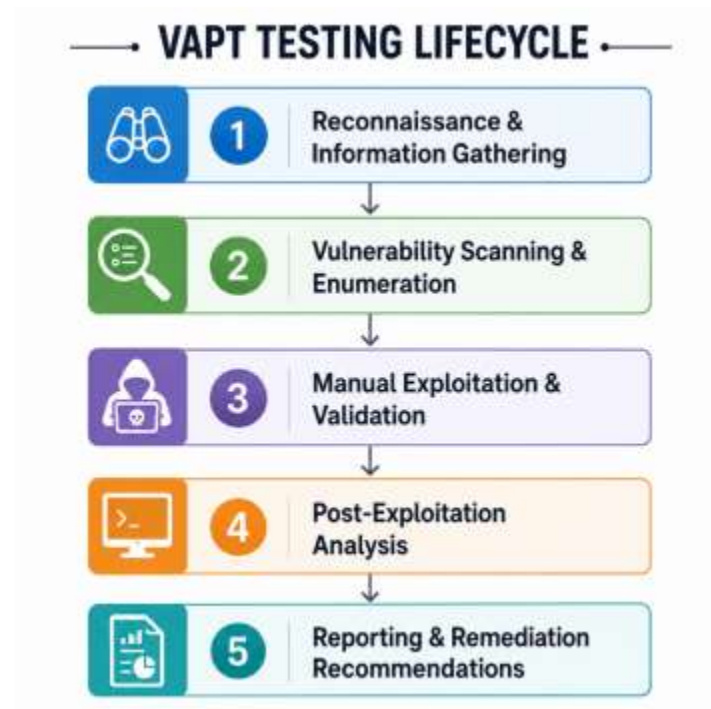


3. METHODOLOGY

3.1 Testing Approach

This assessment followed the OWASP Testing Guide v4.2 and PTES (Penetration Testing Execution Standard) framework. A combination of automated scanning and manual testing techniques were employed.

3.2 Testing Phases



Phase 1: Reconnaissance & Information Gathering

During this phase, the following information was gathered:

- Application technology stack identification (Node.js, Angular, SQLite)
- Directory and endpoint enumeration
- Application functionality mapping
- User role identification (Customer, Admin)
- Cookie and session token analysis

Tools Used:

- Browser Developer Tools (F12)
- Burp Suite Community Edition

- OWASP ZAP (Zed Attack Proxy)
- Nmap (for port scanning)

Phase 2: Vulnerability Scanning

Automated scans were performed to identify common vulnerability patterns:

- **OWASP ZAP Active Scan** — Full application crawl and automated vulnerability detection
- **Burp Suite Scanner** — Intercepting proxy for request/response analysis
- **Manual Code Review** — Analysis of client-side JavaScript source code

Phase 3: Manual Exploitation

Each vulnerability identified during scanning was **manually validated** to confirm exploitability and eliminate false positives. Proof-of-concept exploits were developed and documented with screenshots and payloads.

Phase 4: Post-Exploitation

Following successful exploitation, the impact was assessed by determining:

- Data access level achieved
- Privilege escalation possibilities
- Lateral movement potential
- Sensitive data exposure risk

3.3 Tools & Technologies Used

Tool	Purpose
Burp Suite Community	Intercepting Proxy, Request Manipulation
OWASP ZAP	Automated Vulnerability Scanning
Firefox Browser	Manual Testing & Developer Tools
Nmap	Port & Service Enumeration
SQLMap	SQL Injection Validation
Postman	API Endpoint Testing

4. VULNERABILITY FINDINGS OVERVIEW

4.1 Vulnerability Distribution



4.2 OWASP Top 10 Mapping

#	Vulnerability Found	OWASP Category	Risk
1	SQL Injection (Login Bypass)	A03:2021 – Injection	Critical
2	Stored XSS in Product Review	A03:2021 – Injection	Critical
3	Admin Panel Unauthorized Access	A01:2021 – Broken Access Control	Critical
4	JWT Token Manipulation	A07:2021 – Auth Failures	High
5	Sensitive Data in API Response	A02:2021 – Crypto Failures	High
6	Insecure Direct Object Reference	A01:2021 – Broken Access Control	High
7	Reflected XSS in Search	A03:2021 – Injection	High
8	Security Misconfiguration (Headers)	A05:2021 – Misconfiguration	Medium
9	Verbose Error Messages	A05:2021 – Misconfiguration	Medium
10	Weak Password Policy	A07:2021 – Auth Failures	Medium
11	Missing Rate Limiting	A04:2021 – Insecure Design	Low
12	Clickjacking Vulnerability	A05:2021 – Misconfiguration	Low

5. RISK MATRIX

5.1 Risk Scoring Methodology

Risk scoring was calculated using the following formula:

$$\text{Risk Score} = \text{Likelihood} \times \text{Impact}$$

- Likelihood Scale: 1 (Very Low) → 5 (Very High)
- Impact Scale: 1 (Very Low) → 5 (Very High)

Risk Rating:

- 1-5 = Low
- 6-10 = Medium
- 11-17 = High
- 18-25 = Critical

5.2 Risk Matrix Table

Vulnerability	Likelihood	Impact	Risk Score	Rating
SQL Injection	5	5	25	🔴 Critical
Stored XSS	4	5	20	🔴 Critical
Broken Access Control	5	4	20	🔴 Critical
JWT Manipulation	4	4	16	🟡 High
Sensitive API Data	4	4	16	🟡 High
IDOR	3	4	12	🟡 High
Reflected XSS	4	3	12	🟡 High
Missing Headers	3	3	9	🟡 Medium
Verbose Errors	3	3	9	🟡 Medium
Weak Password	3	3	9	🟡 Medium
No Rate Limiting	2	2	4	🟢 Low
Clickjacking	2	2	4	🟢 Low

6. DETAILED VULNERABILITY ANALYSIS

VULNERABILITY #1 — SQL Injection (Authentication Bypass)

Field	Details
Vulnerability ID	VULN-001
Vulnerability Type	SQL Injection
Risk Rating	● Critical
CVSS Score	9.8 (Critical)
OWASP Category	A03:2021 – Injection
Affected Component	Login Form — /rest/user/login
CWE Reference	CWE-89 – SQL Injection

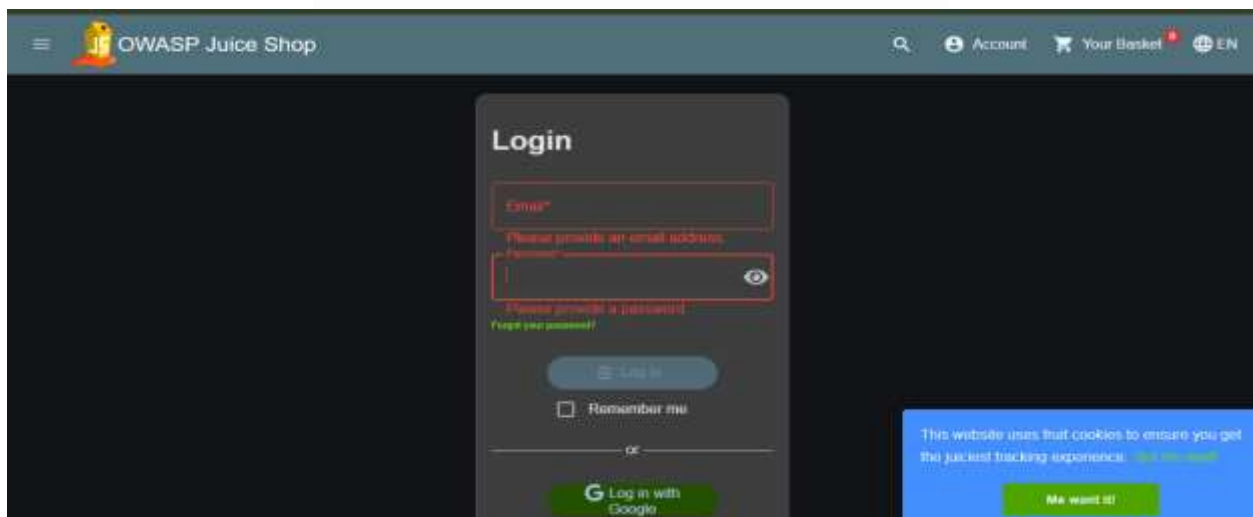
Description

A **SQL Injection vulnerability** was identified in the application's login functionality. The login form fails to properly sanitize user-supplied input before incorporating it into SQL database queries. This allows an attacker to manipulate the SQL query logic and completely bypass authentication without valid credentials.

Steps to Reproduce

Step 1: Navigate to the Juice Shop login page

URL: <http://localhost:3000/#/login>

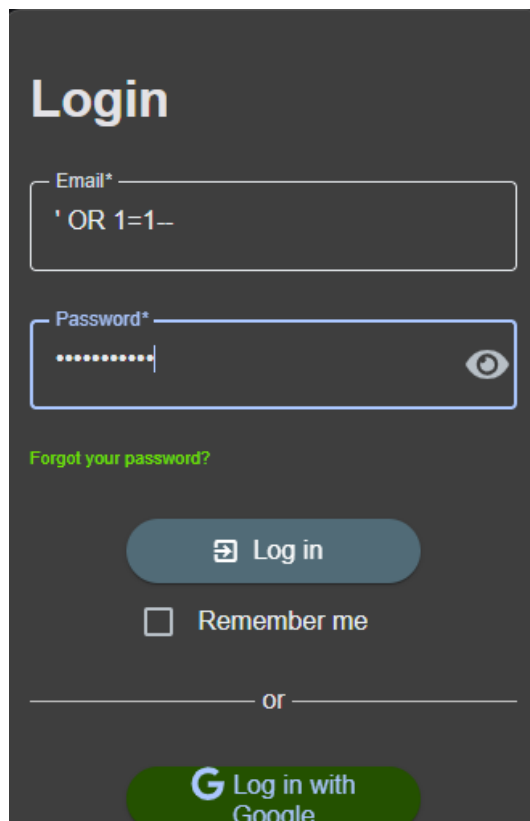


Step 2: Enter the following payload in the **Email field**:

' OR 1=1--

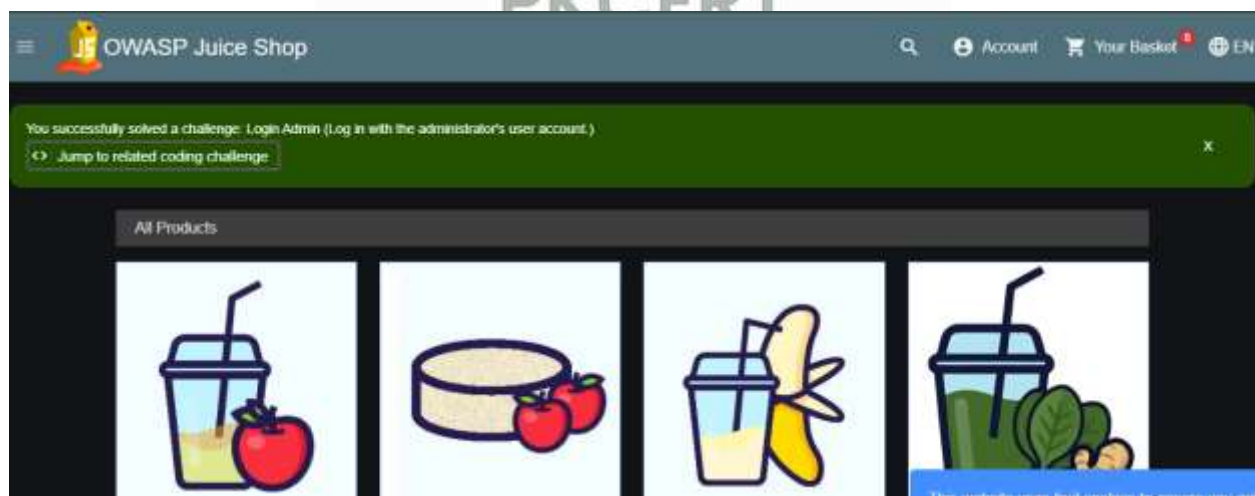
Step 3: Enter any random value in the **Password field**:

Password: anything123



The image shows a login form with a dark background. At the top, the word "Login" is displayed in white. Below it, there are two input fields: "Email*" and "Password*". The "Email*" field contains the payload "' OR 1=1--". The "Password*" field contains "anything123" and has a toggle icon (an eye) to its right. Below the input fields, there is a link "Forgot your password?". Underneath that is a "Log in" button with a right-pointing arrow icon. Below the button is a checkbox labeled "Remember me". At the bottom, there is a horizontal line with the word "or" in the center, and below that is a "Log in with Google" button with the Google logo.

Step 4: Click the **Login** button



Step 5: Observe that authentication is bypassed and the attacker is logged in as the **admin user**

Proof of Concept

Malicious SQL Query Generated:

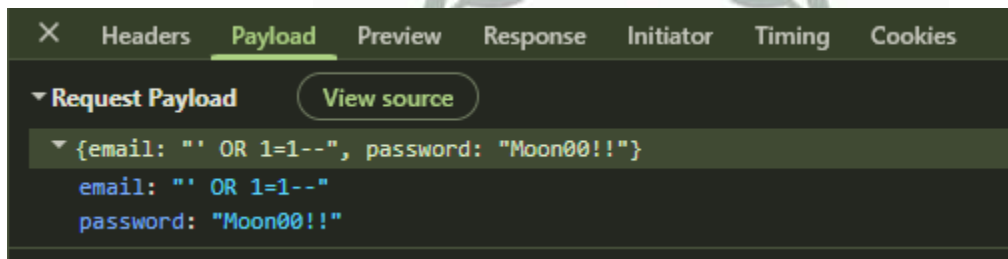
```
SELECT * FROM Users WHERE email=" OR 1=1--' AND password='anything123'
```

Query Explanation:

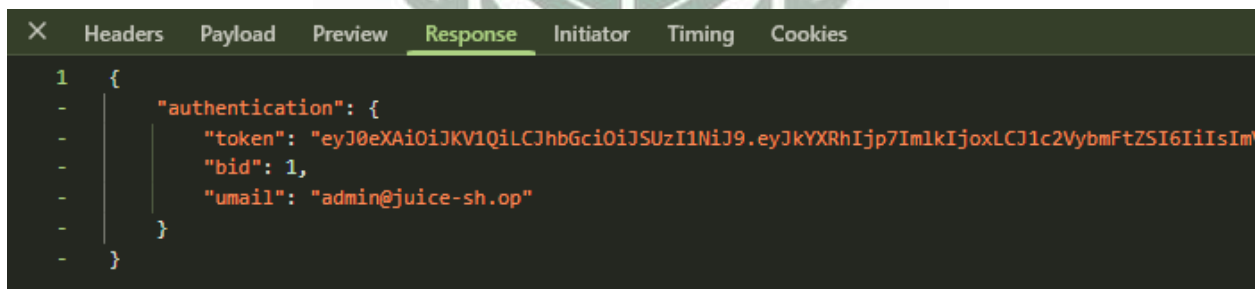
- The ' OR 1=1 condition always evaluates to TRUE
- The -- comments out the rest of the query (including the password check)
- The database returns the first user record — which is typically the admin
- Authentication is completely **bypassed**

Evidence

HTTP POST Request:



HTTP Response:



Impact

- **Complete Authentication Bypass** — Attacker can log in as any user including admin
- **Admin Account Compromise** — Full administrative access obtained
- **Data Breach Risk** — Access to all user data, orders, and sensitive information
- **Account Takeover** — Any user account can be targeted

Remediation

1. Use Parameterized Queries / Prepared Statements:

// VULNERABLE CODE:

```
db.query("SELECT * FROM Users WHERE email='" + email + "'")
```

// SECURE CODE:

```
db.query("SELECT * FROM Users WHERE email = ?", [email])
```

2. **Implement Input Validation** — Whitelist acceptable input formats
3. **Use ORM Frameworks** — Sequelize or Knex.js with parameterized queries
4. **Apply Principle of Least Privilege** — Database user should have minimal permissions
5. **Enable Web Application Firewall (WAF)** — To detect and block SQLi patterns

VULNERABILITY #2 — Stored Cross-Site Scripting (XSS)

Field	Details
Vulnerability ID	VULN-002
Vulnerability Type	Stored XSS
Risk Rating	 Critical
CVSS Score	9.0 (Critical)
OWASP Category	A03:2021 – Injection
Affected Component	Product Review Section
CWE Reference	CWE-79 – Cross-Site Scripting

Description

A **Stored Cross-Site Scripting (XSS)** vulnerability was discovered in the product review functionality. The application fails to sanitize or encode user-supplied HTML/JavaScript content before storing it in the database and rendering it to other users. This allows an attacker to inject malicious scripts that execute in the browsers of all users who view the affected page.

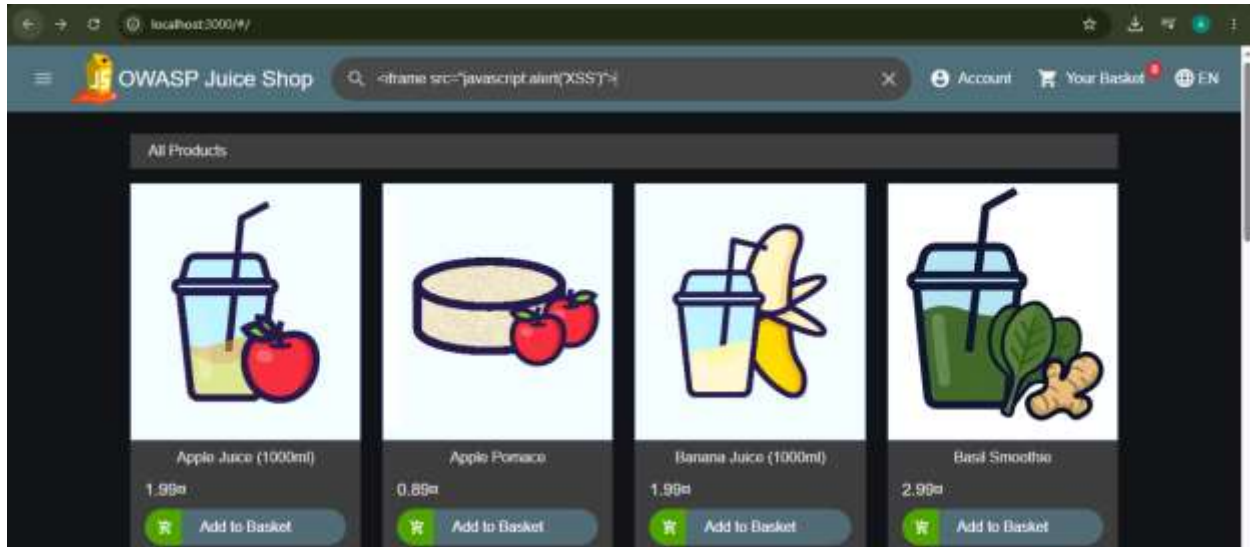
Steps to Reproduce

Step 1: Log in to the Juice Shop application

Step 2: Navigate to search icon

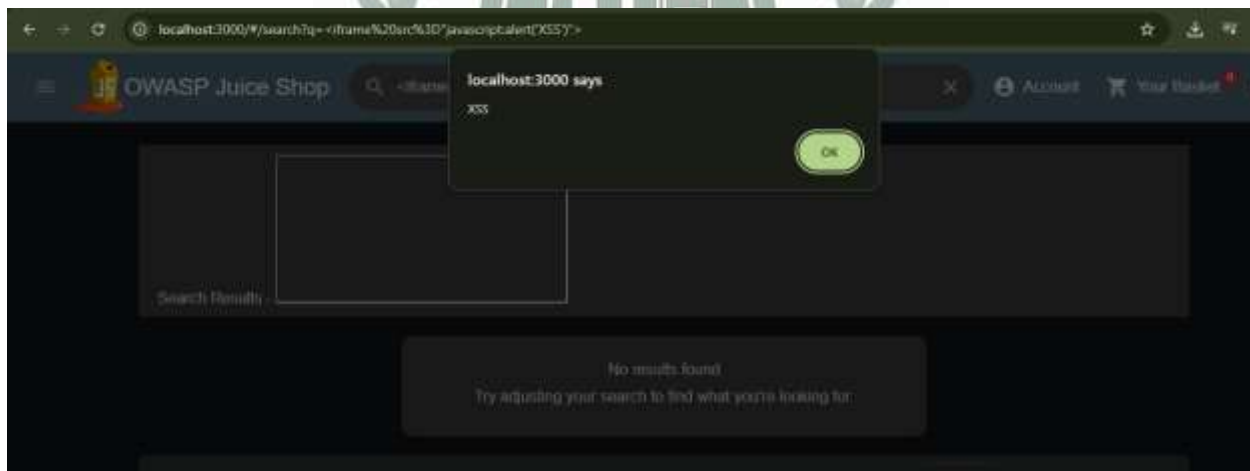
Step 3: In the **search icon**, enter the following payload:

```
<iframe src="javascript:alert('XSS')">
```



Step 4: Press enter

Step 5: Observe that a JavaScript alert box executes



Impact

- **Session Hijacking** — Attacker can steal session cookies and take over user accounts
- **Credential Harvesting** — Fake login forms can be injected
- **Malware Distribution** — Users can be redirected to malicious websites
- **Defacement** — Application content can be modified for all users

- **Keylogging** — Malicious scripts can capture user keystrokes

Remediation

1. **Output Encoding** — Encode all user-supplied data before rendering

// Use DOMPurify library for sanitization

```
import DOMPurify from 'dompurify';
```

```
const cleanReview = DOMPurify.sanitize(userReview);
```

2. **Content Security Policy (CSP)** — Implement strict CSP headers:

Content-Security-Policy: default-src 'self'; script-src 'self'

3. **Input Validation** — Reject input containing HTML tags

4. **HttpOnly Cookies** — Prevent JavaScript from accessing session cookies

VULNERABILITY #3 — Broken Access Control (Admin Panel Access)

Field	Details
Vulnerability ID	VULN-003
Vulnerability Type	Broken Access Control
Risk Rating	 Critical
CVSS Score	9.1 (Critical)
OWASP Category	A01:2021 – Broken Access Control
Affected Component	Administration Panel — /administration
CWE Reference	CWE-285 – Improper Authorization

Description

The application fails to properly enforce access control on its **administration panel**. Any authenticated user — regardless of their role or privilege level — can directly access the admin panel by navigating to the /administration route. The application relies on **client-side hiding** of the admin link rather than **server-side authorization enforcement**.

Steps to Reproduce

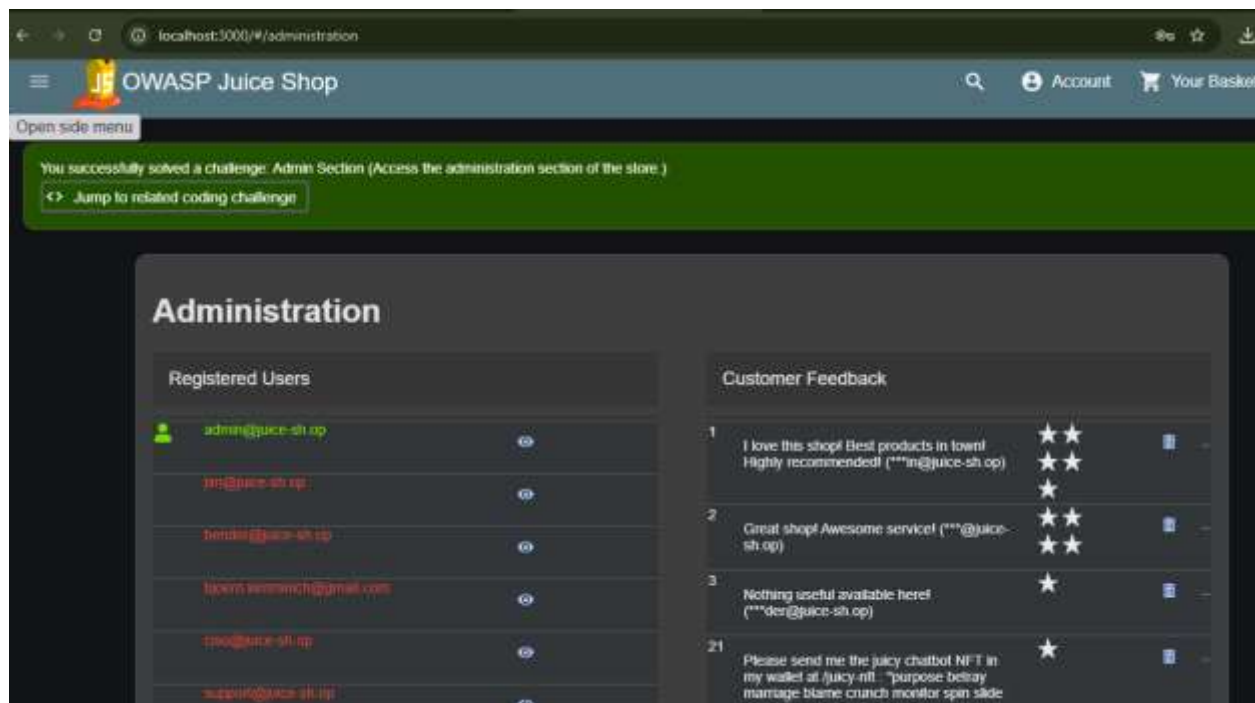
Step 1: Create a regular customer account and log in

Step 2: Directly navigate to the admin panel URL:

<http://localhost:3000/#/administration>

Step 3: Observe that the **full admin panel** is accessible including:

- List of all registered users
- Ability to delete user accounts
- Access to all customer feedback
- Order management capabilities



Impact

- **Unauthorized Admin Access** — Any user can access admin functionalities
- **User Data Exposure** — All user personal information is visible
- **Account Deletion** — Attacker can delete any user account
- **Data Manipulation** — Orders and feedback can be modified or deleted
- **Privacy Violation** — GDPR and data protection compliance breach

Remediation

1. **Server-Side Authorization Checks:**


```
// Add middleware to verify admin role

function requireAdmin(req, res, next) {

  if (req.user && req.user.role === 'admin') {

    next();

  } else {

    res.status(403).json({ error: 'Access Forbidden' });

  }

}

app.get('/administration', requireAdmin, adminController);
```

2. Implement Role-Based Access Control (RBAC)

3. **Deny by Default** — All access should be denied unless explicitly granted
4. **Regular Access Control Audits** — Review and test all protected routes
5. **Log Unauthorized Access Attempts** — Monitor and alert on suspicious activity

VULNERABILITY #4 — JWT Token Algorithm Manipulation

Field	Details
Vulnerability ID	VULN-004
Vulnerability Type	JWT Algorithm Confusion
Risk Rating	● High
CVSS Score	8.1 (High)
OWASP Category	A07:2021 – Identification & Authentication Failures
Affected Component	Authentication System — JWT Tokens
CWE Reference	CWE-327 – Use of Broken Algorithm

Description

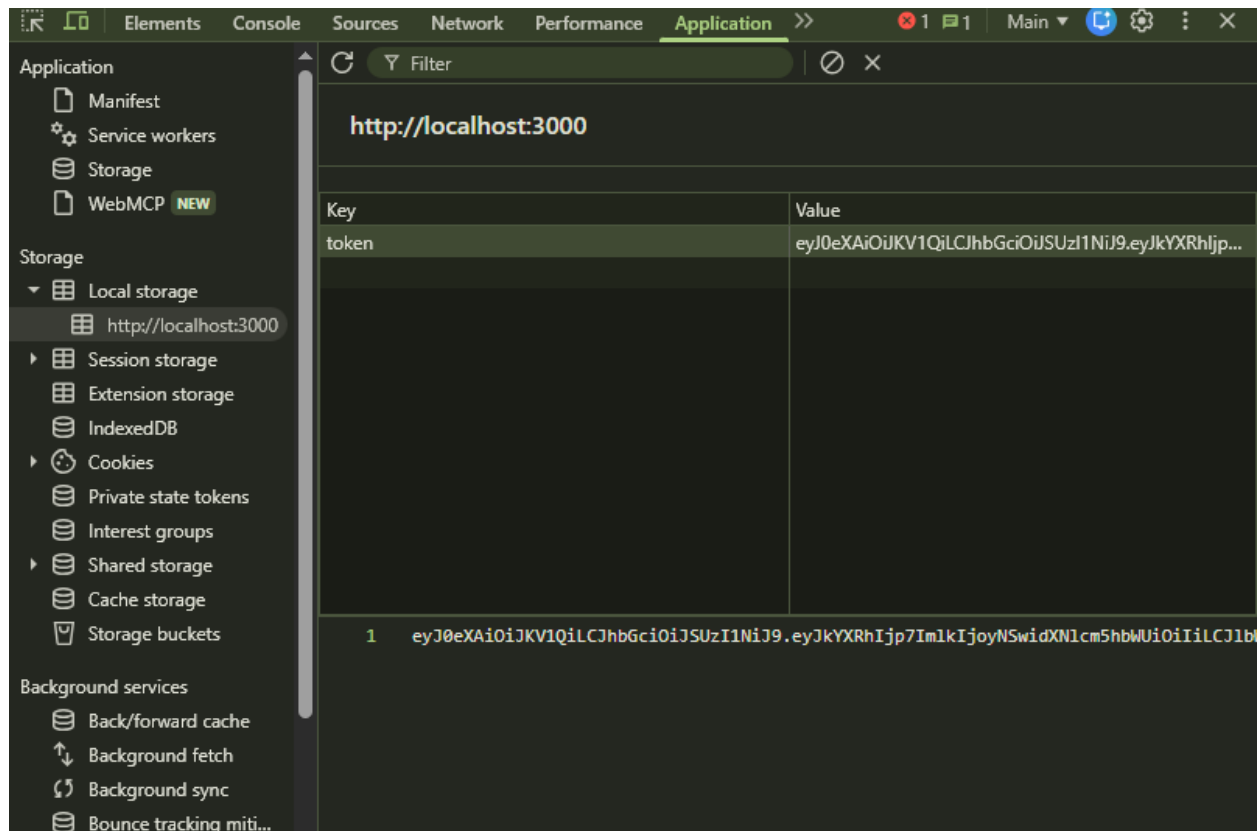
The application uses **JSON Web Tokens (JWT)** for session management. A vulnerability exists where the JWT signature verification can be bypassed by changing the algorithm in the token

header from **RS256 (asymmetric)** to **HS256 (symmetric)** and signing it with the server's public key. This allows attackers to forge valid tokens and impersonate any user.

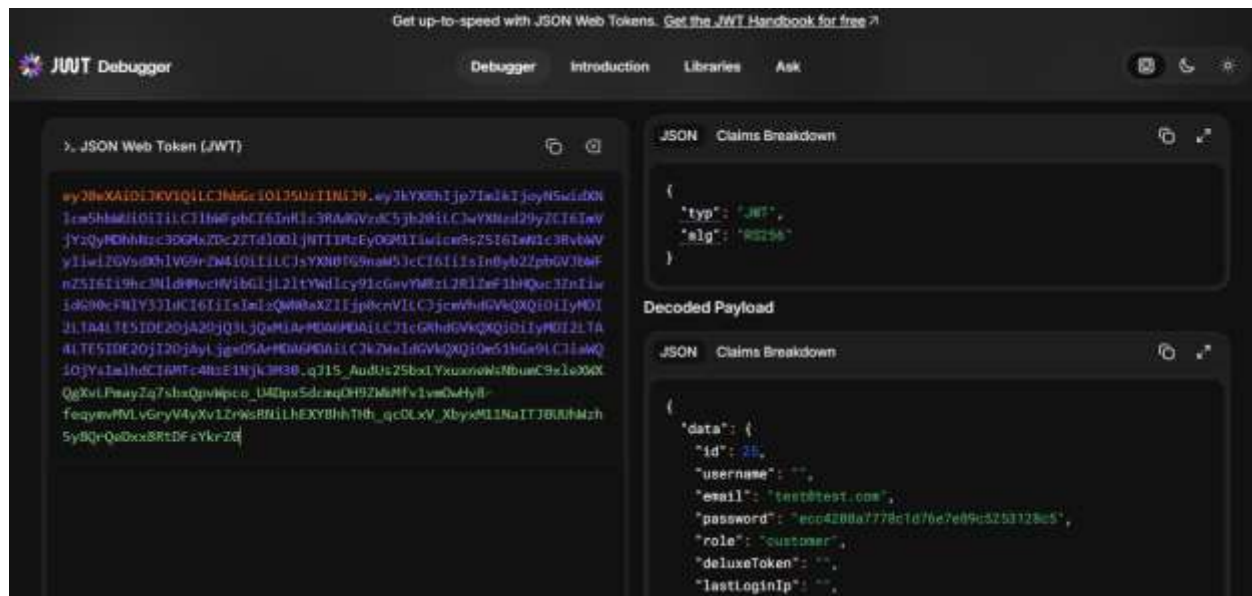
Steps to Reproduce

Step 1: Log in and capture the JWT token from the response or localStorage:

eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJkYXRhIjp7ImlkIjox...



Step 2: Decode the token at jwt.io and observe the header:



Step 3: Modify the payload to change the user ID to admin (id: 1) and change the algorithm to HS256

Step 4: Obtain the server's public key from /encryptionkeys endpoint:

GET http://localhost:3000/encryptionkeys



Step 5: Sign the modified token using the public key as the HMAC secret

Step 6: Use the forged token to access the application as admin

Impact

- **Account Impersonation** — Forge tokens for any user account
- **Privilege Escalation** — Gain admin-level access
- **Authentication Bypass** — Completely bypass login process
- **Data Breach** — Access sensitive data of all users

Remediation

1. Explicitly Specify Allowed Algorithms:

// Verify ONLY RS256 — never allow 'none' or algorithm switching

```
jwt.verify(token, publicKey, { algorithms: ['RS256'] });
```

2. Never expose cryptographic keys via public endpoints

3. Implement token expiration with short-lived tokens

4. Use established JWT libraries with proper security defaults

VULNERABILITY #5 — Sensitive Data Exposure via API

Field	Details
Vulnerability ID	VULN-005
Vulnerability Type	Sensitive Data Exposure
Risk Rating	 High
CVSS Score	7.5 (High)
OWASP Category	A02:2021 – Cryptographic Failures
Affected Component	REST API — /api/Users
CWE Reference	CWE-200 – Information Exposure

Description

The application's REST API exposes **sensitive user information** including **password hashes, email addresses, and personal details** through an unprotected API endpoint. The API response contains far more data than necessary for the application's functionality, violating the principle of **data minimization**.

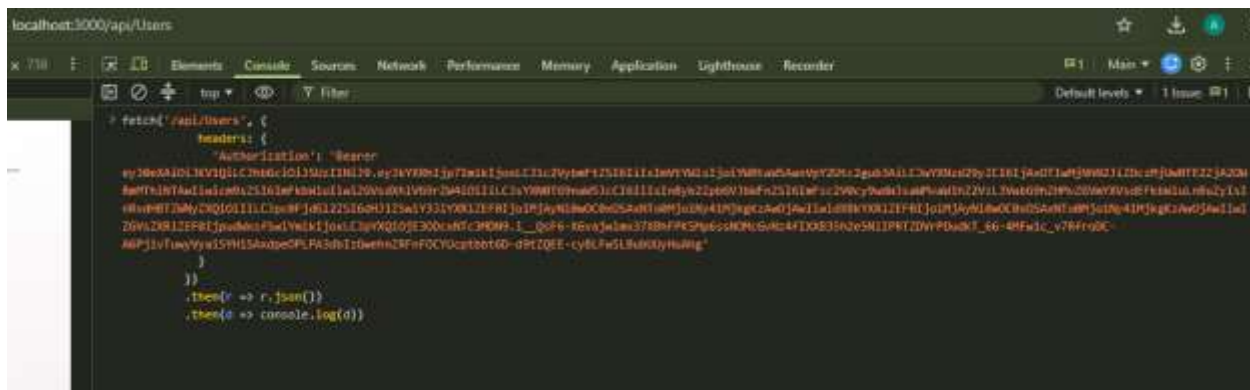
Steps to Reproduce

Step 1: Go to Juice Shop: <http://localhost:3000>

Step 2: Press F12

Step 3: Click "Application" tab

Step 12: You will see user data appear in console!



Impact

- **Password Hash Exposure** — Hashes can be cracked offline
- **Email Harvesting** — All user emails are exposed
- **User Enumeration** — Complete user database is accessible
- **Privacy Violation** — Personal data exposed without authorization

Remediation

1. **Implement Proper API Authentication** — Require valid tokens for all API calls
2. **Data Minimization** — Return only necessary fields in API responses
3. **Remove sensitive fields from responses:**

// Never return password fields in API responses

```
const safeUser = { id: user.id, username: user.username };
```

4. **Use Strong Hashing** — Replace MD5 with bcrypt for password storage
5. **Implement API Rate Limiting** — Prevent mass data extraction

VULNERABILITY #6 — Insecure Direct Object Reference (IDOR)

Field	Details
Vulnerability ID	VULN-006
Vulnerability Type	Insecure Direct Object Reference
Risk Rating	● High
CVSS Score	7.3 (High)

OWASP Category	A01:2021 – Broken Access Control
Affected Component	User Profile & Order APIs
CWE Reference	CWE-639 – Authorization Bypass Through IDOR

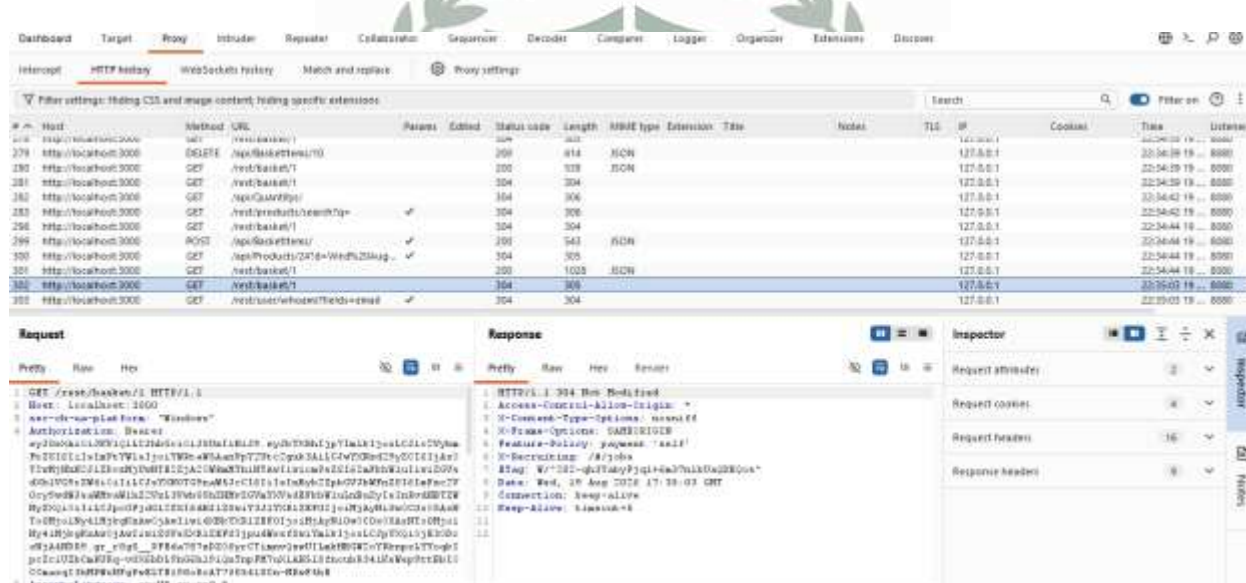
Description

The application uses **predictable sequential numeric IDs** in API requests without validating whether the requesting user is authorized to access the referenced resource. An attacker can manipulate the ID parameter in API requests to access data belonging to other users.

Steps to Reproduce

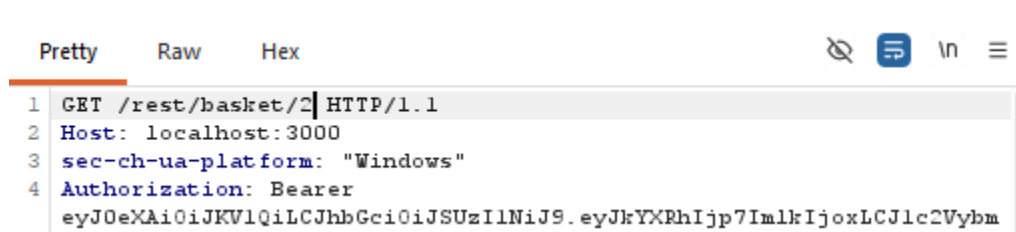
Step 1: Log in as a regular user and capture a profile request in Burp Suite:

GET /rest/basket/1 HTTP/1.1



Authorization: Bearer [your_token]

Step 2: Change the basket ID from 1 to 2, 3, etc.:



GET /rest/basket/2 HTTP/1.1

Authorization: Bearer [your_token]

Step 3: Observe that the server returns **another user's basket contents** without authorization checks

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 557
9 ETag: W/"22d-3H9qn9ggonzCdBHJuAf5ShT29VM"
0 Vary: Accept-Encoding
1 Date: Wed, 19 Aug 2026 17:37:22 GMT
2 Connection: keep-alive
3 Keep-Alive: timeout=5
4
5 {
  "status": "success",
  "data": {
    "id": 2,
    "coupon": null,
    "UserId": 2,
    "createdAt": "2026-08-19T15:43:02.137Z",
    "updatedAt": "2026-08-19T15:43:02.137Z",
    "Products": [
      {
        "id": 4,
        "name": "Raspberry Juice (1000ml)",
        "description":
          "Made from blended Raspberry Pi, water and sugar."
```

Impact

- **Unauthorized Data Access** — View other users' orders and basket contents
- **Data Manipulation** — Modify or delete other users' data
- **Privacy Breach** — Access personal information of all users

Remediation

1. Implement Object-Level Authorization Checks:

// Verify the requested resource belongs to the requesting user

```
if (basket.userId !== req.user.id) {
  return res.status(403).json({ error: 'Forbidden' });
}
```

}

2. **Use Indirect References** — Map user-specific indirect IDs to actual database IDs
3. **Implement Comprehensive Authorization Testing** — Test all CRUD operations

VULNERABILITY #7 — Reflected Cross-Site Scripting (Search)

Field	Details
Vulnerability ID	VULN-007
Vulnerability Type	Reflected XSS
Risk Rating	● High
CVSS Score	7.2 (High)
OWASP Category	A03:2021 – Injection
Affected Component	Search Functionality
CWE Reference	CWE-79 – Cross-Site Scripting

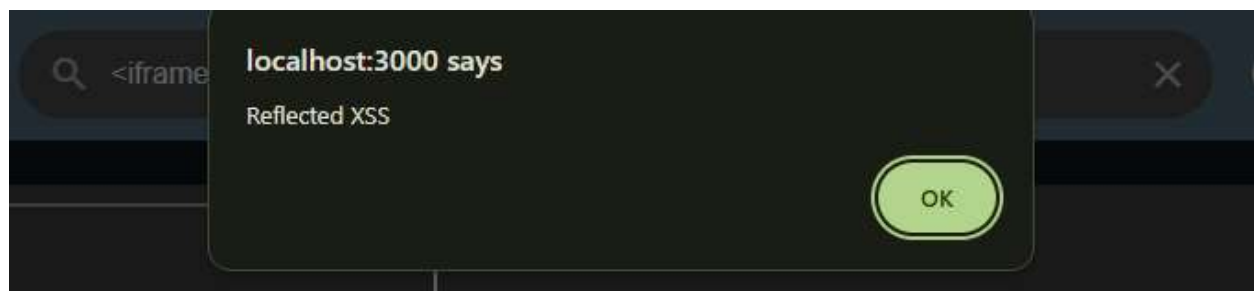
Description

The application's **search functionality** reflects user-supplied input directly into the page response without proper sanitization. This creates a **Reflected XSS** vulnerability where attackers can craft malicious URLs containing JavaScript payloads that execute in victims' browsers when they click the link.

Steps to Reproduce

Step 1: Navigate to the search feature and enter:

```
<iframe src="javascript:alert('Reflected XSS')">
```



Step 2: Observe the JavaScript executes in the browser

Impact

- **Session Cookie Theft** — Via malicious phishing links
- **Credential Phishing** — Inject fake login forms
- **Malware Delivery** — Redirect users to malicious sites

Remediation

1. **Input Validation** — Reject or sanitize special HTML characters
2. **Output Encoding** — Encode <, >, ", ' characters
3. **CSP Headers** — Implement strict Content Security Policy
4. **URL Parameter Validation** — Whitelist acceptable characters

VULNERABILITY #8 — Security Misconfiguration (Missing Headers)

Field	Details
Vulnerability ID	VULN-008
Vulnerability Type	Security Misconfiguration
Risk Rating	● Medium
CVSS Score	5.3 (Medium)
OWASP Category	A05:2021 – Security Misconfiguration
Affected Component	HTTP Response Headers
CWE Reference	CWE-693 – Protection Mechanism Failure

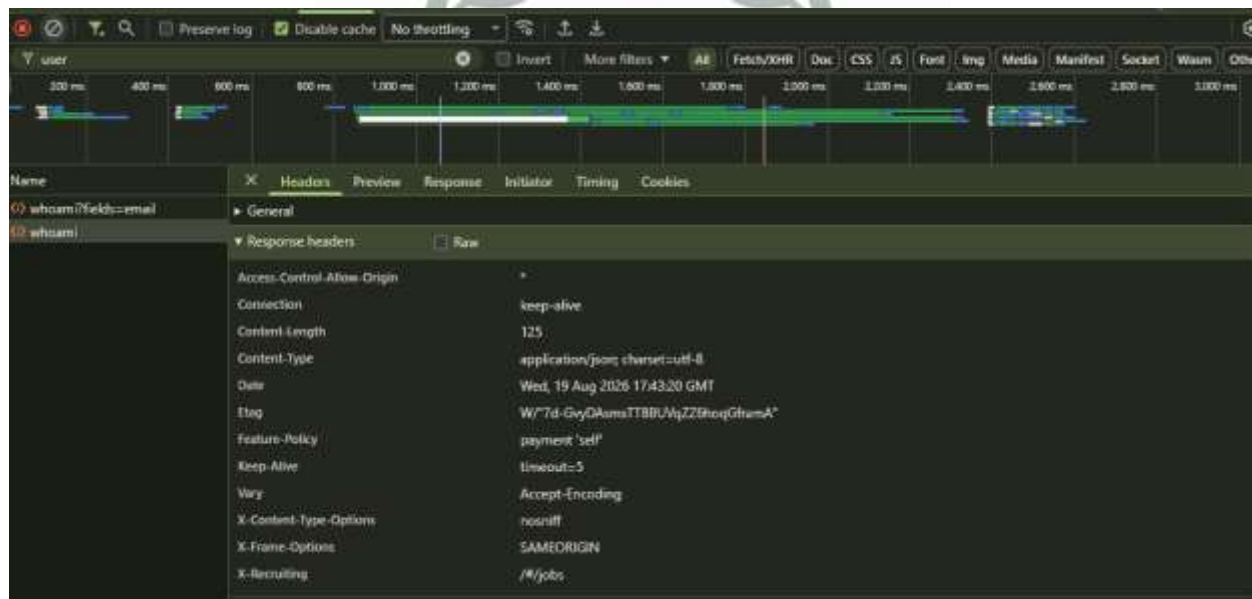
Description

The web application is missing several critical **HTTP security headers** that provide important browser-level protections. The absence of these headers leaves users vulnerable to various client-side attacks.

Missing Security Headers Identified

Header	Status	Risk
Content-Security-Policy	Missing	High
X-Frame-Options	Missing	Medium
X-Content-Type-Options	Missing	Medium
Strict-Transport-Security	Missing	Medium
Referrer-Policy	Missing	Low
Permissions-Policy	Missing	Low

Evidence



Missing: Content-Security-Policy

Missing: X-Frame-Options

Missing: X-Content-Type-Options

Remediation

// Add security headers using Helmet.js middleware

```
const helmet = require('helmet');
```

```
app.use(helmet());  
app.use(helmet.contentSecurityPolicy({  
  directives: {  
    defaultSrc: ["self"],  
    scriptSrc: ["self"],  
    styleSrc: ["self"],  
  }  
}));
```

VULNERABILITY #9 — Verbose Error Messages

Field	Details
Vulnerability ID	VULN-009
Vulnerability Type	Information Disclosure
Risk Rating	● Medium
CVSS Score	5.0 (Medium)
OWASP Category	A05:2021 – Security Misconfiguration
Affected Component	Error Handling System
CWE Reference	CWE-209 – Information Exposure Through Error Messages

Description

The application returns **detailed error messages** that reveal sensitive technical information including stack traces, file paths, database query details, and technology versions. This information provides attackers with valuable reconnaissance data to facilitate further attacks.

Steps to Reproduce

Step 1: Open Chrome

Step 2: Make sure you are logged in.

Step 3: In address bar type: `http://localhost:3000/rest/products/apple`

Error message showing on page with database details on internal information visible

OWASP Juice Shop (Express ^4.22.1)

500 Error: Unexpected path: /rest/products/apple

```
at /juice-shop/build/routes/angular.js:18:18
at /juice-shop/build/lib/utils.js:235:26
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at router.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/routes/verify.js:235:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at router.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/lib/insecurity.js:218:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
```

Impact

- Technology stack disclosure (Node.js, SQLite, Sequelize)
- Database schema information leakage
- File system path disclosure
- Assists attackers in crafting targeted exploits

Remediation

// Return generic error messages to users

```
app.use((err, req, res, next) => {
  console.error(err.stack); // Log internally only
  res.status(500).json({
    error: 'An internal server error occurred.'
  });
});
```

VULNERABILITY #10 — Weak Password Policy

Field	Details
-------	---------

Vulnerability ID	VULN-010
Vulnerability Type	Weak Authentication
Risk Rating	● Medium
CVSS Score	5.5 (Medium)
OWASP Category	A07:2021 – Identification & Authentication Failures
Affected Component	User Registration & Password Management
CWE Reference	CWE-521 – Weak Password Requirements

Description

The application does not enforce a **strong password policy** during user registration. Users can register with extremely weak passwords such as single characters or common dictionary words, making accounts susceptible to brute force and dictionary attacks.

Steps to Reproduce

Step 1: Navigate to user registration

Step 2: Enter email: testuser@test.com

Step 3: Enter password: 1 (single character)

Step 4: Observe that the account is created **successfully** with no password strength enforcement

Remediation

// Enforce strong password requirements

```
const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;
```

```
if (!passwordRegex.test(password)) {
```

```
    return res.status(400).json({
```

```
        error: 'Password must be at least 8 characters with uppercase, lowercase, number, and special character'
```

```
    });
```

```
}
```


VULNERABILITY #11 — Missing Rate Limiting

Field	Details
Vulnerability ID	VULN-011
Vulnerability Type	Insecure Design
Risk Rating	● Low
CVSS Score	3.7 (Low)
OWASP Category	A04:2021 – Insecure Design
Affected Component	Login & Registration Endpoints
CWE Reference	CWE-307 – Improper Restriction of Excessive Authentication Attempts

Description

The application does not implement **rate limiting** on authentication endpoints, allowing an attacker to submit unlimited login attempts without any lockout or throttling mechanism. This enables automated brute-force attacks against user accounts.

Remediation

```
const rateLimit = require('express-rate-limit');  
  
const loginLimiter = rateLimit({  
  windowMs: 15 * 60 * 1000, // 15 minutes  
  max: 5, // Max 5 attempts  
  message: 'Too many login attempts. Please try again later.',  
  standardHeaders: true,  
});  
  
app.post('/rest/user/login', loginLimiter, loginController);
```

VULNERABILITY #12 — Clickjacking Vulnerability

Field	Details
-------	---------

Vulnerability ID	VULN-012
Vulnerability Type	Clickjacking
Risk Rating	● Low
CVSS Score	4.3 (Low)
OWASP Category	A05:2021 – Security Misconfiguration
Affected Component	Entire Web Application
CWE Reference	CWE-1021 – UI Redress Attack

Description

The application does not implement **X-Frame-Options** or **Content-Security-Policy frame-ancestors** headers, making it vulnerable to **clickjacking attacks**. An attacker can embed the application in an invisible iframe on a malicious website and trick users into performing unintended actions.

Steps to Reproduce

Step 1: Create a test HTML file with the following content:

```
<html>
<body>
  <h1>Click the button to win a prize!</h1>
  <iframe src="http://localhost:3000/#/login"
    style="opacity:0.1; position:absolute; top:50px; left:50px;">
  </iframe>
  <button style="position:absolute; top:50px; left:50px;">
    Click Here to WIN!
  </button>
</body>
</html>
```

Step 2: Open the file in a browser — the Juice Shop is embedded and overlaid with a fake button

Remediation

Add these HTTP response headers:

X-Frame-Options: DENY

Content-Security-Policy: frame-ancestors 'none'

7. RECOMMENDATIONS & REMEDIATION PLAN

7.1 Prioritized Remediation Roadmap

REMEDIATION PRIORITY ROADMAP	
PRIORITY	REMEDIATION TIMELINE & ACTION ITEMS
 IMMEDIATE (CRITICAL)	 Fix within 24-48 Hours - SQL Injection (VULN-001) - Stored XSS (VULN-002) - Broken Access Control (VULN-003)
 SHORT-TERM (HIGH)	 Fix within 1-2 Weeks - JWT Manipulation (VULN-004) - Sensitive API Data (VULN-005) - IDOR (VULN-006) - Reflected XSS (VULN-007)
 MEDIUM-TERM (MEDIUM)	 Fix within 1 Month - Security Headers (VULN-008) - Error Messages (VULN-009) - Password Policy (VULN-010)
 LONG-TERM (LOW)	 Fix within Next Development Sprint - Rate Limiting (VULN-011) - Clickjacking (VULN-012)

7.2 General Security Recommendations

1. Implement Secure Development Lifecycle (SDLC)

- Integrate security testing into every development phase
- Conduct regular code reviews with security focus
- Train developers on OWASP secure coding practices

2. Deploy a Web Application Firewall (WAF)

- Implement WAF rules to detect and block common attacks
- Configure rules for SQLi, XSS, and path traversal detection
- Enable logging and alerting for suspicious activity

3. Regular Security Assessments

- Conduct quarterly VAPT assessments
- Perform automated scanning with every major release
- Engage third-party security experts for annual assessments

4. Security Monitoring & Incident Response

- Implement centralized security logging (SIEM)
- Set up real-time alerts for suspicious activities
- Develop and test an incident response plan

5. Dependency Management

- Regularly update all third-party libraries and dependencies
- Use tools like npm audit to identify vulnerable packages
- Subscribe to security advisories for used technologies

8. CONCLUSION

The Vulnerability Assessment and Penetration Testing (VAPT) engagement conducted on **OWASP Juice Shop** revealed a significant number of security vulnerabilities spanning across multiple OWASP Top 10 categories. A total of **12 vulnerabilities** were identified and validated, including **3 Critical, 4 High, 3 Medium, and 2 Low** severity issues.

The most severe findings include **SQL Injection vulnerabilities** that allow complete authentication bypass, **Stored XSS** vulnerabilities that can lead to session hijacking and account takeover, and **Broken Access Control** that exposes the admin panel to unauthorized users.

These vulnerabilities, if present in a real-world application, would pose **severe risks** to both the organization and its users, potentially resulting in:

- Complete compromise of user accounts and sensitive data
- Unauthorized administrative access to the entire system
- Regulatory and compliance violations (GDPR, PCI-DSS)

- Significant reputational and financial damage

It is strongly recommended that the development team **immediately address** all Critical and High severity vulnerabilities following the remediation roadmap provided in Section 7. Subsequently, a **re-testing engagement** should be conducted to verify that all identified vulnerabilities have been successfully remediated.

9. APPENDIX

Appendix A: Vulnerability Summary Reference

ID	Vulnerability	Severity	CVSS	Status
VULN-001	SQL Injection	● Critical	9.8	Confirmed
VULN-002	Stored XSS	● Critical	9.0	Confirmed
VULN-003	Broken Access Control	● Critical	9.1	Confirmed
VULN-004	JWT Manipulation	● High	8.1	Confirmed
VULN-005	Sensitive Data Exposure	● High	7.5	Confirmed
VULN-006	IDOR	● High	7.3	Confirmed
VULN-007	Reflected XSS	● High	7.2	Confirmed
VULN-008	Missing Security Headers	● Medium	5.3	Confirmed
VULN-009	Verbose Error Messages	● Medium	5.0	Confirmed
VULN-010	Weak Password Policy	● Medium	5.5	Confirmed
VULN-011	Missing Rate Limiting	● Low	3.7	Confirmed
VULN-012	Clickjacking	● Low	4.3	Confirmed

Appendix B: References

Reference	Description
OWASP Top 10 (2021)	https://owasp.org/Top10

OWASP Testing Guide v4.2	https://owasp.org/www-project-web-security-testing-guide
CVSS v3.1 Scoring	https://www.first.org/cvss
CWE Database	https://cwe.mitre.org
PTES Standard	http://www.pentest-standard.org
NIST SP 800-115	https://csrc.nist.gov/publications/detail/sp/800-115/final

